Vienna Routing: Cost and Heuristics Documentation

Vienna Traffic Router Team

April 22, 2026

This document explains how distances are measured, how edge costs ($g(n)$) are calculated, and how heuristics ($h(n)$) are combined to guide search algorithms like A*.

1 Distance Calculation

All distance measurements in the Vienna project use the **Haversine Formula**. This calculates the great-circle distance between two points defined by their latitude and longitude.

1.1 Formula

The distance d is calculated as follows:

$$
\begin{aligned}
a &= \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \\
c &= 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \\
d &= R \cdot c
\end{aligned}
$$

Where $R = 6,371,000$ meters.

1.2 Implementation Source

In `src/graph/builder.py`:

```python
def haversine(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    R = 6_371_000
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlam / 2) ** 2
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
```

2 Edge Cost ($g(n)$)

The edge cost, or path cost from the start to the current node ($g(n)$), represents the actual weight assigned to an edge in the graph.

2.1 Effective Cost Calculation

1. **Manual Traffic Overrides**:
    - **Blocked**: If intensity ≥ 95, the edge is considered impassable (∞ cost).

- **Sub-blocked**: Multiplier $= 0.5 + (\text{intensity}/100.0) \cdot 2.5$.

2. **Road Quality Multiplier**: Adjusted based on road type and surface.

2.2 Implementation Source

In `src/graph/loader.py` → `get_effective_edge_cost()`:

```
    if overrides_map:
        edge_key = f'{edge["from"]}_{edge["to"]}'
        if edge_key in overrides_map:
            intensity = overrides_map[edge_key]
            if intensity >= 95:
                return float("inf")
            cost *= 0.5 + (intensity / 100.0) * 2.5

    if params is not None and params.get("use_road_quality", True):
        from src.heuristics.road_quality import get_road_quality_multiplier
        cost *= get_road_quality_multiplier(edge, params.get("vehicle_type"))
```

3 Heuristic Cost ($h(n)$)

The heuristic cost ($h(n)$) is an estimate of the remaining cost from the current node to the goal.

3.1 Composite Multiplier

The final $h(n)$ is:

$$h(n) = \text{Haversine}(n, \text{goal}) \cdot \text{Multiplier}_{\text{Total}}$$

4 Admissibility and Capping

4.1 Admissibility

A heuristic is **admissible** if it never overestimates the true cost. The **Haversine distance** is the base admissible heuristic. However, multiplying it by factors > 1.0 (like weather penalties) can make it **inadmissible**.

4.2 The MAX_HEURISTIC_CAP

To prevent the search from becoming purely greedy, the final heuristic is capped:

$$h(n)_{\text{final}} = \min(h, h_{\text{base}} \cdot \text{MAX_HEURISTIC_CAP})$$

The default cap is **50.0**. This ensures **bounded-suboptimality** (ϵ-admissibility), meaning the resulting path is guaranteed to be within a known factor of the optimal path.

4.3 How Capping Helps

- **Search Focus**: Prevents $h(n)$ from ballooning and dwarfing $g(n)$.
- **Greedy Prevention**: Ensures A* doesn't blindly follow low h values.
- **Stability**: Maintains consistent performance when multiple multipliers are stacked.

4.4 Implementation Source

In `src/heuristics/combined.py`:

```
MAX_HEURISTIC_CAP = float(os.getenv("MAX_HEURISTIC_CAP", "50.0"))
# ...
        h = h_base * mult
        # Cap total h against arbitrary blow-up, keep epsilon-admissibility
        return min(h, h_base * MAX_HEURISTIC_CAP)
```

5 Distance and Time Results

5.1 Implementation Source

In `src/algorithms/base.py`:

```
def sum_path_distance(graph: dict, path: list[str]) -> float:
    # ... loops through path and sums graph["edges"][edge_idx]["distance_m"] ...

VEHICLE_SPEEDS = {
    "car":        8.33,   # ~30 km/h urban
    "motorcycle": 9.72,   # ~35 km/h
}
```